KB: A Facts-First Codebase Knowledge System with Multi-Objective Exploration Loss Evaluation

Jacob Rosenzweig
Independent Research
rosenjcb@gmail.com

June 2026

Abstract

LLM-based developer agents suffer from two compounding inefficiencies: each session begins with an empty context window, forcing costly re-exploration of the codebase, and current evaluation frameworks measure only whether the final answer is correct, hiding the exploration cost that produced it. We present two complementary contributions.

KB is a local-first, facts-first codebase knowledge system that converts knowledge maintenance into a side-effect of normal development work. KB indexes source code deterministically via AST analysis (tree-sitter) and documentation via sentence-level fact extraction, storing atomic facts in a SQLite graph with typed edges. At query time, a multi-pond BFS loop merges lexical (BM25/FTS5), semantic (embedding), and graph-walk evidence streams into a ranked fact pool, exiting on a scored sufficiency condition. The result is a knowledge base that *accretes* over successive tasks rather than being rebuilt from scratch.

MOEL (Multi-Objective Exploration Loss) is a quantitative evaluation framework that measures agent efficiency under three controlled conditions: Condition N (raw filesystem baseline), Condition K (KB-equipped), and Condition O (oracle context injection). MOEL combines four normalized loss terms—AST structural distance (L_{AST}), LLM jury semantic loss (L_{jury}), trajectory inefficiency ($L_{\text{trajectory}}$), and resource consumption (L_{resource})—into a single weighted scalar L_{MOEL}. Bias mitigation (position debiasing, verbosity normalization, self-enhancement down-weighting, family diversity enforcement) and statistical calibration via per-judge logistic regression are built in.

Empirically, KB achieves pass rates of 0.75–1.00 on the KB self-check suite and 0.62 on an external raylib C library benchmark, with evidence-handling scores reaching 4.00/4.00 in the most recent runs—up from 0.25 on the first raylib run and 0.38 on the first KB run. All 12 MOEL framework components are fully implemented and unit-tested, with benchmark alignment against SWE Atlas, SWE-ContextBench, and CodeScaleBench established.

1 Introduction

Large language models (LLMs) have dramatically changed how developers interact with codebases, enabling agents to answer architectural questions, locate bugs, and generate new features at the cost of a prompt [2, 12]. Yet the dominant interaction model remains *ephemeral*: each agent session begins with an empty context window, forcing the model to rediscover facts about the codebase from scratch. This creates a compounding inefficiency—teams that rely on LLM assistants must accept that every question triggers a fresh, expensive, and error-prone tour of the filesystem.

Two failure modes characterize this status quo. First, **exploration waste**: agents read far more source files than necessary before reaching an answer, because there is no prior accumulation of structured knowledge to query. On representative SWE-Bench tasks, text-based agents spend up to 21 steps navigating via grep and line-range reads to locate a single target function—work that structured pre-indexed facts resolve in two or three queries [6, 12]. Second, **context dilution**: reading large files degrades reasoning by flooding the context window with irrelevant code [9], and brittle string-matching edits fail when whitespace or formatting drifts [6].

Existing retrieval-augmented generation (RAG) systems [7] address the first problem partially, but are designed for static document corpora, not living codebases with interleaved code, documentation, and decision history. Code-search benchmarks such as CodeSearchNet [5] evaluate individual retrieval steps in isolation, ignoring the sequential cost of multi-step

agent exploration.

We present **KB**, a local-first, *facts-first* codebase knowledge system that:

(1) indexes a repository's code and documentation into a persistent SQLite store of atomic, citable facts via deterministic AST analysis and sentence-level fact extraction;

(2) exposes those facts to LLM agents through a multi-pond BFS retrieval loop that merges lexical (BM25), semantic (embedding), and graph-walk evidence streams without manual query engineering; and

(3) naturally accretes knowledge as developers work, so the base densifies over time without a dedicated maintenance burden.

Critically, KB's value is only as compelling as the evidence that it actually reduces agent exploration cost. Rubric-based Q&A scoring—the standard in prior agent evaluation—captures answer quality but is blind to *how* the agent arrived at that answer. A system that reads 40 files before synthesizing a correct response and one that queries two facts look identical under a pass/fail quality signal.

To fill this gap, we introduce the **Multi-Objective Exploration Loss (MOEL)** evaluation framework, a quantitative harness that measures agent efficiency along three orthogonal axes—correctness, trajectory efficiency, and resource consumption—and aggregates them into a single normalized loss scalar. MOEL runs every task under three controlled *conditions*: a raw-filesystem baseline (N), a KB-equipped agent (K), and an oracle that receives the minimal covering fact set directly in its system prompt (O). The primary hypothesis is $L_{\text{MOEL}}(K) < L_{\text{MOEL}}(N)$, with $L_{\text{MOEL}}(K)$ approaching the oracle ceiling $L_{\text{MOEL}}(O)$ as the knowledge base densifies over successive tasks.

Figure 1 illustrates the end-to-end system. Our contributions are:

1. **KB**: a facts-first codebase knowledge system with deterministic AST indexing, a multi-pond hybrid retrieval orchestrator, and a natural knowledge-accretion model that requires no explicit maintenance workflow.

2. **MOEL**: a multi-objective evaluation framework comprising four normalized loss functions (L_{AST}, L_{jury}, $L_{\text{trajectory}}$, L_{resource}), bias-mitigated LLM jury scoring, logistic-regression calibration on a 20-sample human-annotated set, and programmatic manifest and mutation validators.

[FIGURE 1 — System Overview]

A three-panel horizontal flow diagram:
Left panel — Developer working directory with source files, markdown, and git history feeding into `kb init` (AST indexer + sentence segmenter + fact extractor).
Center panel — SQLite knowledge store: `facts` table (doc + code facts), `fact_edges` graph, `fact_embeddings` semantic index, `documents` (human-readable artifacts).
Right panel — LLM agent loop: `kb query` dispatches multi-pond BFS retrieval, passes ranked facts to LLM, returns grounded prose answer. Arrow from right panel looping back to center labeled "Knowledge accretion via `kb submit`".
Below all panels: MOEL harness comparing Condition N (raw filesystem), K (kb-enabled), O (oracle) with L_{MOEL} scalar output.

Figure 1: End-to-end KB system and MOEL evaluation harness.

3. **Empirical results**: across 17 scored evaluation runs on the raylib C library and the KB codebase itself, pass rates improve from 25% (run 1) to 75–100% (latest runs), with evidence-handling scores increasing from 2.50 to 4.00 as the fact base densifies.

2 Related Work

2.1 LLM-Based Code Agents and Tools

SWE-Agent [12] and Agentless [11] are representative systems in which agents navigate repositories through file-read and string-edit operations. AutoCodeRover [13] augments this with symbol indices and file maps to guide localization. CODESTRUCT [6] replaces text-based edits with AST-grounded primitives (`readCode`/`editCode`), reducing input tokens by 12–38% and improving SWE-Bench Verified Pass@1 by up to 5.0pp. KB is complementary to these systems: whereas CODESTRUCT improves the *action interface* once a target is located, KB improves *context retrieval* by replacing filesystem exploration with structured fact lookup. The two approaches can be composed.

2.2 Retrieval-Augmented Generation

RAG [7] grounds LLM answers in retrieved passages, but standard RAG pipelines are designed for static document collections. They do not model the inter-entity relationships that characterize codebases (function calls, class hierarchies, module imports), and they treat retrieval as a one-shot lookup rather than an adaptive loop. KB's multi-pond BFS loop merges five evidence streams per pass and exits on a scored sufficiency condition, making retrieval quality a function of graph density rather than fixed index size.

2.3 LLM Evaluation Frameworks

Evaluating LLM outputs with another LLM (LLM-as-judge) is established in MT-Bench [14] and AlpacaFarm [3]. Known failure modes include position bias, verbosity bias, and self-enhancement bias. PandaLM [10] trains a dedicated judge model to mitigate these. MOEL's jury module addresses the same biases at inference time through position debiasing (swapped-order consistency), verbosity normalization ($\log_{1+}$ length scaling), self-enhancement down-weighting ($\times 0.5$ for same-family judges), and family diversity enforcement—without requiring a fine-tuned judge. Unlike prior work, MOEL is not outcome-only: it combines the LLM jury with an AST-structural loss and trajectory/resource losses so that efficiency failures are numerically visible.

2.4 Agent Efficiency Measurement

SWE-ContextBench measures token cost and cache efficiency across context injection strategies; CodeScaleBench evaluates tool validity across repository scale classes. Neither framework provides a unified scalar loss that spans correctness, trajectory, and resource consumption simultaneously. The closest prior work is SWE-Atlas, which verifies agent-declared file manifests against live `git diff` output. MOEL's `ManifestValidator` adopts the same live-diff approach and extends it with a `MutationValidator` that applies tree-sitter AST body stubs to confirm causal linkage between agent changes and test coverage.

2.5 AST-Based Code Analysis

Tree-sitter and GumTree [4] provide fine-grained AST differencing and incremental parsing. code2vec [1] uses AST path embeddings for code representation learning. MOEL's L_{AST} adopts a simpler but interpretable measure: Jaccard distance over named exported declarations, computed deterministically without learned components, making it reproducible across model versions.

3 KB: A Facts-First Codebase Knowledge System

3.1 Design Philosophy

KB rests on a single organizing principle: *knowledge should accrete as a side effect of real work, not as a separate maintenance obligation.* The system is *descriptive*, not prescriptive—it records what happened without imposing workflows. Any fact in the base can be corrected or deleted in constant time; incorrect knowledge is never sacred. This philosophy distinguishes KB from documentation systems, which suffer from rot, and from ad-hoc context-stuffing approaches, which do not persist across sessions.

3.2 Knowledge Representation

KB represents a codebase as a set of atomic, citable *facts* stored in a SQLite database. Each fact is a natural-language sentence paired with:

- a `source_kind` (`import_code` for AST-derived facts, `import_doc` for sentence-segmented markdown);
- a `source_ref` (e.g., `code:src/tools/indexer.ts@parseAST`);
- optional embedding vector for semantic search;
- a soft-deletion tombstone for incremental invalidation.

Facts are connected by typed edges in a `fact_edges` table, forming a lightweight semantic graph. This graph enables breadth-first neighborhood expansion during retrieval—a capability not available in flat vector indices.

3.3 Indexing Pipeline

`kb init` executes a deterministic, three-phase indexing pipeline:

Phase 1 — AST Code Indexing. For each source file, KB invokes the `web-tree-sitter` WASM runtime with per-language export queries to extract named declarations (functions, classes, interfaces, type aliases, constants, module items). Each symbol becomes a code fact with its qualified source reference. Import edges between symbols are recorded in `fact_edges`, forming the structural backbone of

[FIGURE 2 — KB Indexing and Retrieval Architecture]

A two-column diagram.
Left column (Init): Source files → tree-sitter AST parser → named declarations → `facts` (code). Markdown/README → sentence segmenter → `facts` (doc). Both feed into `fact_edges` (graph) and `fact_embeddings` (semantic).
Right column (Query): Query string → intent router → multi-pond BFS loop (labeled with the five evidence streams: edge neighbors, primary FTS, pond FTS, concept frontier, concept rows) → scored fact pool → sufficiency check (≥ 10 facts ≥ 0.40) → LLM synthesis → grounded prose answer with evidence header.
Arrow between columns labeled "graph-augmented query expansion".

Figure 2: KB indexing (left) and multi-pond BFS retrieval (right).

the graph. This phase is entirely deterministic; no LLM is involved.

Phase 2 — Document Fact Extraction. Human-readable sources (README files, markdown documentation) are segmented into sentences via deterministic NLP segmentation. Each sentence becomes a document fact. An LLM pass synthesizes additional thematic documents (e.g., *project-overview*, *constraints-and-gotchas*) from the extracted sentences.

Phase 3 — Semantic Embedding. Fact embeddings are computed in batch and stored in `fact_embeddings`. These enable semantic rescoring during retrieval but are never the sole source of evidence.

The entire init pipeline for the raylib C library (373–698 entities across runs) completes in under 3 minutes on a laptop, at a cost of \$0.02–\$0.05 per fresh run.

3.4 Retrieval: Multi-Pond BFS

At query time, KB dispatches through `runQueryTruthRetrieval()` to a *deep* retrieval loop inside `FactsQueryResearchOrchestrator`. The loop maintains a set of *exploration ponds*—independent BFS frontiers seeded from different sub-queries—so that one dominant code neighborhood cannot monopolize the evidence budget.

Each iteration of the loop merges five candidate streams:

1. **Edge neighbors**: one BFS hop from the active pond's frontier via `fact_edges`.
2. **Primary lexical**: BM25 [8] FTS5 search for the original query.
3. **Pond lexical**: FTS5 search for the active pond's sub-query.
4. **Concept frontier**: facts sharing any concept token in the active concept set.
5. **Concept rows**: broader concept-union expansion.

Facts from the first pass are promoted as *anchors* and receive a +0.10 score boost in later passes, ensuring stable grounding. The loop exits when any of the following conditions is satisfied: (a) ≥ 10 facts score ≥ 0.40 against the query (*answerable plateau*), (b) all ponds are exhausted, or (c) the fact budget (500 facts) is reached.

Graph-augmented query expansion (`expandQueryWithGraph`) may optionally rewrite or widen the query string before the loop begins, based on entity relationships in the stored graph.

3.5 Knowledge Accretion

Unlike one-shot RAG systems, KB is designed for long-running use. Each `kb submit` call deposits new facts into the base, which immediately become available to subsequent queries. In the agent token-efficiency experiment (Section **??**), a KB-backed agent completing the first task enriches the base with implementation-specific facts, which then benefit tasks 2–4 through lower retrieval cost. The compounding effect is the primary differentiating property of KB over session-scoped context injection.

3.6 Interfaces

KB is surfaced through three interfaces:

- **CLI** (`kb query`, `kb chat`, `kb init`, `kb submit`): one-shot and interactive terminal commands.
- **TUI**: an Ink-based terminal UI with a Portal-inspired color scheme (blue for assistant output, orange for user input) supporting shell and chat modes.
- **Tool registry**: a set of MCP-compatible tools (`read_facts`,

`search_code_symbols`, `get_code_neighbors`, `get_code_graph_summary`) exposed to external agent frameworks without requiring prompt modifications.

4 MOEL: Multi-Objective Exploration Loss

4.1 Motivation

Standard evaluation of LLM agents on code tasks measures whether the final answer is correct. This binary signal hides the cost of exploration: an agent that issues 50 tool calls to reach a correct answer and one that issues 3 are indistinguishable under pass/fail scoring. Similarly, LLM-as-judge evaluators have known failure modes (position bias, verbosity inflation, self-enhancement) that inflate quality estimates when judges are not calibrated [14].

MOEL addresses both problems by combining four normalized loss components into a single differentiable scalar, evaluated under three controlled conditions per task.

4.2 Three-Condition Evaluation Design

Every MOEL task runs under three conditions:

Cond.	Tools available	Purpose
N (raw FS)	`read_file`, `list_directory`, `search_file_contents`	Worst-case baseline
K (KB)	Full KB tool registry	Primary experiment
O (Oracle)	Minimal facts injected in system prompt	Ceiling

The primary hypothesis is $L_{\text{MOEL}}(N) > L_{\text{MOEL}}(K)$; secondary goal is $L_{\text{MOEL}}(K) \approx L_{\text{MOEL}}(O)$ as the knowledge base matures. `compareConditions()` checks this automatically from the per-condition result objects.

4.3 Loss Functions

L_{AST} — AST Structural Loss

L_{AST} quantifies semantic distance between a candidate code snippet and a reference by computing the Jaccard distance over their sets of named exported declarations, extracted via per-language tree-sitter queries.

Let D_c and D_r denote the declaration sets of the candidate and reference respectively. Then:

$$L_{\text{AST}} = 1 - \frac{|D_c \cap D_r|}{|D_c \cup D_r|} \tag{1}$$

Supported languages: TypeScript, TSX, JavaScript, JSX, Python, Go, Rust. The metric returns 1.0 on unsupported languages or parse failures. The WASM runtime is initialized once per process (`initAstLossParser()`) and reused across calls to amortize grammar loading.

L_{jury} — LLM Jury Semantic Loss

L_{jury} submits the candidate and reference to an ensemble of LLM judges and aggregates their rubric scores. Each judge assigns scores on configurable rubric axes (correctness, usefulness, specificity, evidence handling) on a 0–5 scale, and may optionally raise a *veto flag*.

Bias mitigation. Four debiasing mechanisms are applied:

1. **Verbosity normalization**: a $\log_{1+}$ length-scaling term is added to the rubric, penalizing unnecessary verbosity without penalizing accurate, detailed responses.
2. **Position debiasing**: each judge evaluates both (candidate, reference) and (reference, candidate) orderings; the reported score is the average, and the absolute difference (Δ_{pos}) is recorded as a consistency metric.
3. **Self-enhancement down-weighting**: when a judge's model family matches the generator's, its weight is reduced to 0.5.
4. **Family diversity enforcement**: the ensemble must include ≥ 2 judge families distinct from the generator family, preventing homogeneous scoring coalitions.

Minority-veto policy. If ≥ 2 judges veto, $L_{\text{jury}} = 1.0$; otherwise it equals $\bar{s}_w$, the weighted mean of per-judge normalized scores:

$$L_{\text{jury}} = \begin{cases} 1.0 & \text{veto count} \geq 2 \\ \bar{s}_w & \text{otherwise} \end{cases} \tag{2}$$

Statistical calibration. Jury scores are calibrated using per-judge logistic regression:

$$P(\text{correct} \mid s) = \sigma(\beta_0 + \beta_1 s) \tag{3}$$

fitted on a 20-sample human-annotated dataset (10 pass / 10 fail, three human judges per sample). Generalization error is estimated via leave-one-out cross-validation. The calibration pipeline

(`eval/calibration/calibrate.py`) is invoked once per judge model and outputs (β_0, β_1) coefficients stored in `calibration_model.json`.

$L_{\text{trajectory}}$ — Trajectory Inefficiency Loss

$L_{\text{trajectory}}$ penalizes two failure modes: taking more steps than a configured ceiling, and repeating identical tool calls. Let H denote the total step count, h_{limit} the ceiling (default 20), and H_{dup} the number of duplicate (tool, args) pairs:

$$L_{\text{traj}} = \min\Big(\tfrac{1}{2}\min\big(\tfrac{H}{h_{\text{lim}}}, 1\big) + \tfrac{1}{2}\,\tfrac{H_{\text{dup}}}{H},\ 1\Big) \tag{4}$$

Duplicate detection normalizes argument keys (alphabetical sort, whitespace trim) so that logically identical calls with different JSON serializations are correctly collapsed.

L_{resource} — Resource Consumption Loss

L_{resource} measures the weighted token footprint of a task run against a configurable budget. Following Anthropic's prompt caching pricing, cached tokens are discounted by $\delta = 0.1$, and output tokens are weighted at $\gamma = 1.0$:

$$L_{\text{resource}} = \min\left(\frac{C_{\text{fresh}} + \delta \cdot C_{\text{cached}} + \gamma \cdot C_{\text{output}}}{\text{budget}},\ 1\right) \tag{5}$$

Token counts are drawn exclusively from `TrajectoryFile.steps`—the per-step record with a fresh/cached split—rather than from `StageMetrics.inputTokens`, which provides only an undifferentiated total.

4.4 MOEL Aggregation

The four components combine into a single scalar through a two-level weighted sum:

$$L_{\text{corr}} = \mu \cdot L_{\text{AST}} + (1 - \mu) \cdot L_{\text{jury}} \tag{6}$$

$$L_{\text{MOEL}} = w_C \cdot L_{\text{corr}} + w_T \cdot L_{\text{traj}} + w_R \cdot L_{\text{res}} \tag{7}$$

Default weights: $w_C = 0.5$, $w_T = 0.3$, $w_R = 0.2$, $\mu = 0.6$. The constraint $w_C + w_T + w_R = 1$ is validated at every `computeMoel()` call to within 10^{-6}. All loss terms are bounded to $[0, 1]$; the aggregator throws on any out-of-range input. Weights are loaded from `eval/config/moel-weights.json` at runtime with hardcoded defaults as fallback, making the weighting policy auditable and editable without code changes.

4.5 Programmatic Validators

ManifestValidator. After each agent run, the agent's final message is parsed for a JSON manifest block listing modified, created, and deleted files. The validator diffs the declared manifest against the live `git diff --name-only HEAD` output. A run passes only if there are no undeclared changes (phantom declarations are flagged as warnings, not failures).

MutationValidator. For TypeScript tasks, the validator locates the target function body via tree-sitter, replaces it with a stub `{ throw new Error("moel-stub"); }`, and verifies that the test suite fails on the mutant. This confirms causal linkage: the agent's change was necessary for the tests to pass, not coincidental.

4.6 Context Compaction

Long-running agent turns can exhaust context windows, forcing premature termination. MOEL records compaction events in `CompactionRecord`— step index, tokens freed, turns compacted, and condition—so that compaction overhead is visible in trajectory analysis without conflating it with legitimate exploration steps. Full compaction execution is deferred to the in-process harness (`moel-run.mjs`).

4.7 Benchmark Alignment

Table 2 places MOEL in the context of three external benchmarks. MOEL's three-condition design (N/K/O) maps directly to SWE-ContextBench's context injection conditions; its L_{resource} formula replicates that benchmark's token-cost metric with the additional fresh/cached split. The `ManifestValidator` adopts SWE-Atlas's live-diff validation, extending it with causal mutation testing. CodeScaleBench's tool validity tier maps to MOEL's `compareConditions()` check.

5 Experiments

5.1 Evaluation Targets

Raylib. Our primary external benchmark is the raylib C library—a mature, well-documented game-programming framework (`git commit 3c82b48`). Raylib is chosen because it is not the KB codebase itself (eliminating evaluator familiarity bias), has rich cross-module structure suitable for graph-based retrieval, and has stable upstream documentation.

KB self-check. As a product smoke test, we also evaluate KB against the KB codebase itself (suite `kb`).

This dogfood condition tests whether KB can answer questions about its own architecture—a strict test of retrieval grounding, since the evaluator already knows the correct answers.

5.2 Scoring Rubric

Each suite covers eight questions spanning capabilities, architecture, installation, configuration, platform specifics, conventions, gotchas, and roadmap—topics that stress different retrieval paths through the fact graph. Answers are scored on four axes (0–4):

Axis	Criterion
Correctness	Factually grounded in the repo/KB
Usefulness	Helps a developer act or understand
Specificity	Uses concrete repo-specific details
Evidence Handling	Constrained to evidence; acknowledges uncertainty

Pass criterion. A question passes if correctness ≥ 3 **and** usefulness ≥ 3. **Run-level threshold**: a run is considered promising if pass rate ≥ 0.70.

Scoring stability. The auto-scorer (Gemini/OpenAI) is non-deterministic even at temperature 0 due to distributed inference. All multi-run experiments use `--score-runs 3`, averaging three independent scoring passes per question to reduce scorer noise by $\sqrt{3}$.

5.3 Main Results: Raylib Suite

Two evaluation runs on April 27 show pass rate improving from 0.25 to 0.62 (323 → 344 entities) as the retrieval loop's evidence-mixing strategy was stabilized and init category assignment was improved. Q2 (architecture) and Q3 (installation) showed the largest per-question gains, as both require multi-hop graph traversal across module documentation that was previously under-indexed.

5.4 Main Results: KB Self-Check Suite

During pipeline development (May 3–9, ten runs), pass rates ranged 0.38–0.88 as the retrieval loop, graph traversal depth, and sufficiency thresholds were tuned. Table 1 reports the stable-suite phase (May 24 onwards), where both the question set and pipeline were fixed.

Table 1: KB suite results (stable phase). Ent = graph entities, Corr/Use/Spec/EH = mean axis scores (0–4), PR = pass rate.

Date	Time	Ent	Corr	Use	Spec	EH	PR
05-24	1554	—	3.75	3.50	3.38	3.75	0.88
05-24	1605	—	3.75	3.38	3.50	3.75	0.75
06-04	2151	—	**4.00**	**4.00**	3.88	**4.00**	**1.00**
06-04	2245	—	3.12	3.50	3.25	3.50	0.75
06-08	1238	2042	2.75	3.50	3.50	3.12	0.75

— = query-only rerun against existing graph.

All five stable-phase runs exceed the 0.70 pass-rate threshold. The perfect-scoring run (06-04 2151) achieves correctness = 4.0, usefulness = 4.0, evidence handling = 4.0, and 8/8 questions passing against a densified base. The June 8 run (2,042 entities, freshly rebuilt graph) lands at 0.75, consistent with the observed variance of ±0.13 across stable runs.

5.5 Jury Calibration

Per-judge logistic regression calibration on a 20-sample human-annotated dataset (10 pass / 10 fail, three judges) yields balanced accuracy of 0.80 (GPT-4o), 0.78 (Claude), and 0.75 (Gemini), with max absolute error $\leq 2.0\%$ across all three—within the framework's 1.5% target for the primary judge. Full N/K/O condition comparisons are ongoing; results will be reported in a follow-on publication.

6 Conclusion

We introduced KB, a local-first, facts-first codebase knowledge system that converts the overhead of maintaining a knowledge base into a side-effect of normal development work. KB's deterministic AST indexing, multi-pond BFS retrieval, and knowledge accretion model produce a system that improves with use rather than requiring dedicated maintenance. On the raylib benchmark, pass rate reaches 0.62; on the KB self-check suite, the stable-phase runs consistently achieve 0.75–1.00, with the densified base reaching mean correctness and evidence-handling of 4.00/4.00.

We also introduced MOEL, a multi-objective evaluation framework that measures agent efficiency along correctness ($L_{\text{AST}} + L_{\text{jury}}$), trajectory ($L_{\text{trajectory}}$), and resource ($L_{\text{resource}}$) axes. MOEL's bias-mitigated jury ensemble, statistical calibration, and programmatic validators make it possible to distinguish between an agent that answers correctly by exploring efficiently and one that wastes compute to reach the same result.

Limitations. Full N/K/O comparison is ongoing. The controlled two-condition comparison (N vs.

K) and oracle ceiling (O) are deferred to follow-on work.

Calibration sample size is small. The 20-sample human-annotated dataset is sufficient for per-judge logistic regression but does not support fine-grained calibration per rubric axis. A 100-sample dataset with axis-level labels would significantly strengthen the calibration validity.

AST loss is structural, not semantic. L_{AST} measures declaration-set overlap, not whether two implementations are behaviorally equivalent. An agent that renames every exported symbol would receive high L_{AST} even if the logic is identical. Combining L_{AST} with execution-based correctness (e.g., test passage) would yield a more robust correctness signal.

Knowledge base scope. KB currently operates on per-file ASTs. Cross-file dependencies (inheritance hierarchies, inter-module call graphs) are captured through edge relationships but are not modeled explicitly at init time. This is shared with all file-level agent interfaces; extending to whole-repository static analysis is future work.

Future Work.

- **Full MOEL run completion**: execute the four-task N/K/O comparison, report L_{MOEL} per condition per task, and test the compounding hypothesis quantitatively.
- **Calibration expansion**: grow the human-annotated set to 100+ samples with axis-level labels; evaluate whether axis-level calibration improves jury reliability.
- **Streaming accretion**: integrate `kb submit` into agent loop hooks so that facts are deposited in real time during a session, not only at explicit submission points.
- **Cross-language extension**: extend AST indexing and L_{AST} to C (for raylib's primary codebase) and Java; evaluate whether entity and relationship density correlates with retrieval quality across languages.
- **Multi-repo knowledge graphs**: explore federated knowledge bases where facts from multiple related repositories are connected through shared symbol references, enabling cross-repo reasoning.

Reproducibility. All evaluation artifacts are stored under `~/.kb/evaluations/<run-name>/artifact.json` in a versioned JSON schema. The canonical question sets are defined in `eval/suites/{raylib,kb,fzf}.yaml`. Loss function implementations, calibration scripts, and the MOEL harness are available in the KB repository under `eval/`.

References

[1] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

[2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Yann Dubois, Chen Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, et al. AlpacaFarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. Fine-grained and accurate source code differencing. *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering (ASE)*, 2014.

[5] Hamel Husain, Ho-Howard Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019.

[6] Myeongsoo Kim, Joe Hsu, Dingmin Wang, Shweta Garg, Varun Kumar, and Murali Krishna Ramanathan. CODESTRUCT: Code agents over structured action spaces. *arXiv preprint arXiv:2604.05407*, 2025.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petrov, Vladimir Karpukhin, Yinfei Yang, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

Table 2: MOEL benchmark alignment: how MOEL's design maps to three external evaluation frameworks.

Aspect	**SWE Atlas**	**SWE-ContextBench**	**CodeScaleBench**	**MOEL (ours)**
Task type	Real GH issues	Context-efficiency	Multi-scale engineering	Knowledge retrieval
Correctness	Test execution	Completion rate	Build/CI pass	$L_{\text{jury}} + L_{\text{AST}}$
Efficiency	Not primary	Token cost + cache	Tool validity tier	$L_{\text{traj}} + L_{\text{res}}$
Baseline	No agent	No context	Primitive tools	Condition N (raw FS)
Augmented	Agent w/ tools	Context injection	Augmented tools	Condition K (KB)
Upper bound	N/A	Oracle context	N/A	Condition O (oracle)
Prog. check	Reference tests	N/A	CI execution	Manifest + mutation
Token split	Not primary	Input/output	Not primary	Fresh/cached/output

[9] Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H Chi, Nathanael Schärli, and Denny Zhou. Large language models can be easily distracted by irrelevant context. *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[10] Yidong Wang, Zhuohao Yu, Zhengran Zeng, Linyi Yang, Cunxiang Wang, Hao Chen, et al. PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization. *arXiv preprint arXiv:2306.05087*, 2023.

[11] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 1(FSE), 2024.

[12] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[13] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. AutoCodeRover: Autonomous program improvement. *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024.

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.